

Урок 12. Корутины. Связь между родительской и дочерней корутиной

10 апреля 2020

В этом уроке разбираемся, как создается дочерняя корутина. Рассмотрим, как формируется ее контекст, и как работает ее связь с родительской корутиной.

Перед тем, как обсуждать техническую реализацию связи родительской и дочерней корутин, давайте попробуем увидеть эту связь на примере.

Мы уже несколько раз запускали подобный код:

```
1 val job = scope.launch {
2     log("parent start")
3     launch {
4         log("child start")
5         delay(1000)
6         log("child end")
7     }
8     log("parent end")
9 }
```

И знаем, как он сработает:

```
21:20:42.969 parent start
21:20:42.970 parent end
21:20:42.970 child start
21:20:43.971 child end
```

Код родительской корутины выполняется сразу и не ждет выполнения дочерней. Это, пожалуй, является одним из самых непонятных моментов, когда начинаешь изучать корутины. Т.е. вроде одна корутина родительская, а внутри нее дочерняя, но при этом родительская корутина не ждет дочернюю и никакой связи между ними не видно.

Чтобы увидеть эту связь, мы будем проверять джоб родительской корутины. Метод `job.isActive` подскажет нам статус корутины - еще работает или уже завершена. Фокус в том, что даже если родительская корутина выполнила весь свой код, это еще не значит, что она завершена. Это зависит от того, завершились ли ее дочерние корутины. И именно в этом проявляется связь между родительской и дочерней корутинами.

Давайте дополним пример:

```
1 val job = scope.launch {
2     log("parent start")
3     launch {
4         log("child start")
5         delay(1000)
6         log("child end")
7     }
8     log("parent end")
9 }
10
11 scope.launch {
12     delay(500)
13     log("parent job is active: ${job.isActive}")
14     delay(1000)
15     log("parent job is active: ${job.isActive}")
16 }
```

Будем запускать отдельную корутину, которая в лог пишет статус джоба родительской корутины. Первый раз мы пишем лог, когда родительская корутина уже выполнила свой код, но ее дочерняя корутина еще работает. А второй лог запишется уже после того, как дочерняя завершила работу.

Логи:

```
22:03:03.820 parent start
22:03:03.822 parent end
22:03:03.823 child start
22:03:04.327 parent job is active: true
22:03:04.832 child end
22:03:05.334 parent job is active: false
```

Родительская корутина выполнила весь свой код и начала выполняться дочерняя. Проверяем статус родительской корутины - она все еще активна, хотя код ее завершен. Далее дочерняя корутина завершает работу и мы еще раз проверяем статус родительской корутины - теперь она завершена.

Т.е. чтобы родительская корутина считалась завершенной, ей недостаточно просто выполнить весь свой код. Необходимо также, чтобы завершились все ее дочерние корутины.

Каков практический смысл этого статуса? Где это используется?

Например - при отмене корутин. Вызов метода `cancel` для родительской корутины каскадно отменит и все ее дочерние корутины. Но метод `cancel` сработает только для корутины, которая еще не завершена. Поэтому родительская корутина и ждет завершения дочерних, чтобы мы всегда могли отменить дочерние с помощью родительской.

Еще один пример - метод `join`. В уроке про `launch` мы разбирали этот метод. Он будет ждать, чтобы корутина была именно завершена, а не просто выполнила свой код. Соответственно, если у ожидаемой корутины есть дочерние, то он будет ждать пока они не завершатся.

Создание дочерней корутины

Давайте посмотрим, как связь между корутинами реализована под капотом. Для этого подробно разберем процесс создания и запуска дочерней корутины внутри родительской. Этот процесс состоит из следующих этапов:

1. создание контекста дочерней корутины
2. создание джоба дочерней корутины
3. создание связи с родительской корутиной
4. запуск дочерней корутины (создание Continuation и его отправка в диспетчер)

Билдер выполняет первые два этапа. Далее он вызывает метод `start` у созданного джоба и тот уже сам выполняет третий и четвертый этапы.

Про второй этап говорить особо нечего, т.к. это просто создание экземпляра `*Coroutine` класса, который будет играть роль джоба в корутине. Я упомяну это в первом этапе в моих дальнейших объяснениях.

Четвертый этап мы уже достаточно подробно рассматривали, когда говорили про Continuation и диспетчеры.

Давайте подробно рассмотрим первый и третий этапы.

Как образуется контекст

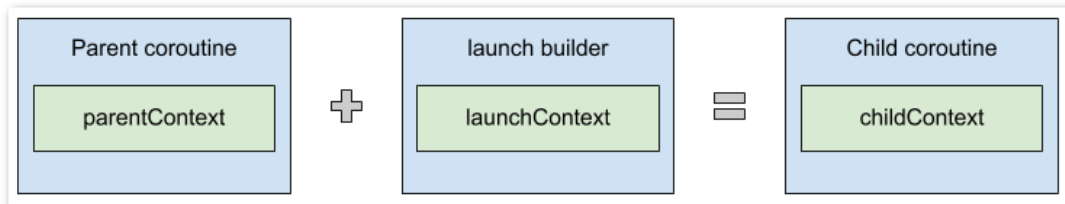
В уроке про контекст мы уже говорили о том, как создается контекст в дочерней корутине. Обсудим это чуть подробнее и со схемой.

Итак, у нас есть такой код:

```
1 | scope.launch { // parent coroutine
2 |
3 |     launch(launchContext) { // child coroutine
4 |
5 |     }
6 |
7 | }
```

Вызов дочерней корутины из родительской и передача в билдер контекста `launchContext`.

Изобразим на схеме обе корутины и билдер:



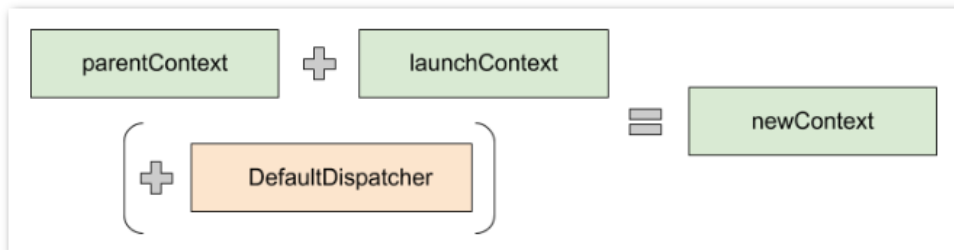
У нас есть родительская корутина с контекстом `parentContext` и контекст `launchContext`, который был передан в билдер `launch`. Из них будет сформирован контекст дочерней корутины - `childContext`. Для этого будет выполнено несколько шагов.

На всякий случай предупреждаю, что используемые мною здесь имена не совпадают с именами переменных в исходниках корутин.

1. К `parentContext` добавляется `launchContext`. При этом происходит замена элементов с одинаковыми ключами. Например, если в `parentContext` был один диспетчер, а в `launchContext` другой, то в итоговом контексте мы получим диспетчер из `launchContext`.

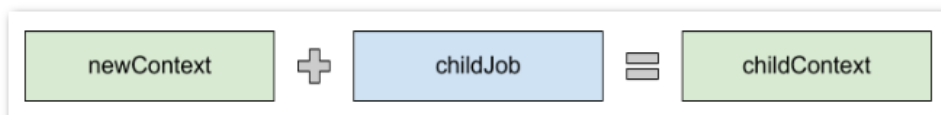
Получившийся контекст назовем `newContext`.

2. Если в `newContext` нет диспетчера, то в него добавляется диспетчер по умолчанию.



3. Создается джоб дочерней корутины. Контекст `newContext` передается в его конструктор.

4. Джоб добавляет к `newContext` сам себя (`this`), получая тем самым новый контекст, который и является итоговым `childContext`. Контекст `newContext` при этом никуда из джоба не делся.



В итоге, в джобе дочерней корутины роль основного контекста играет `childContext`. Но кроме него, там хранится и `newContext`. Он содержит `Job` родителя, который понадобится при создании связи с родительской корутиной.

Создание связи с родительской корутиной

`Job` дочерней корутины достает `Job` родителя (из `newContext`), вызывает его метод `attachChild` и передает туда себя. Родитель создает объект `ChildHandleNode`, и не только оставляет его себе, но и отправляет его же обратно в дочерний `Job`, как результат вызова метода `attachChild`.

Этот объект и является связью. Он хранится в обоих джобах - родительском и дочернем. Для родителя это способ узнать, что есть незавершенные дочерние корутины. А дочерней он нужен, чтобы разорвать связь, когда работа завершена.

Давайте рассмотрим, как эта связь используется в разных случаях, когда родительская корутина вызывает дочернюю. Дочерняя корутина может завершить свой код раньше или позже родительской, и от этого зависит, как взаимодействуют между собой эти корутины.

1. Родительская корутина использует `launch` для запуска дочерней корутины. Дочерняя корутина завершит выполнение своего кода раньше, чем родительская.

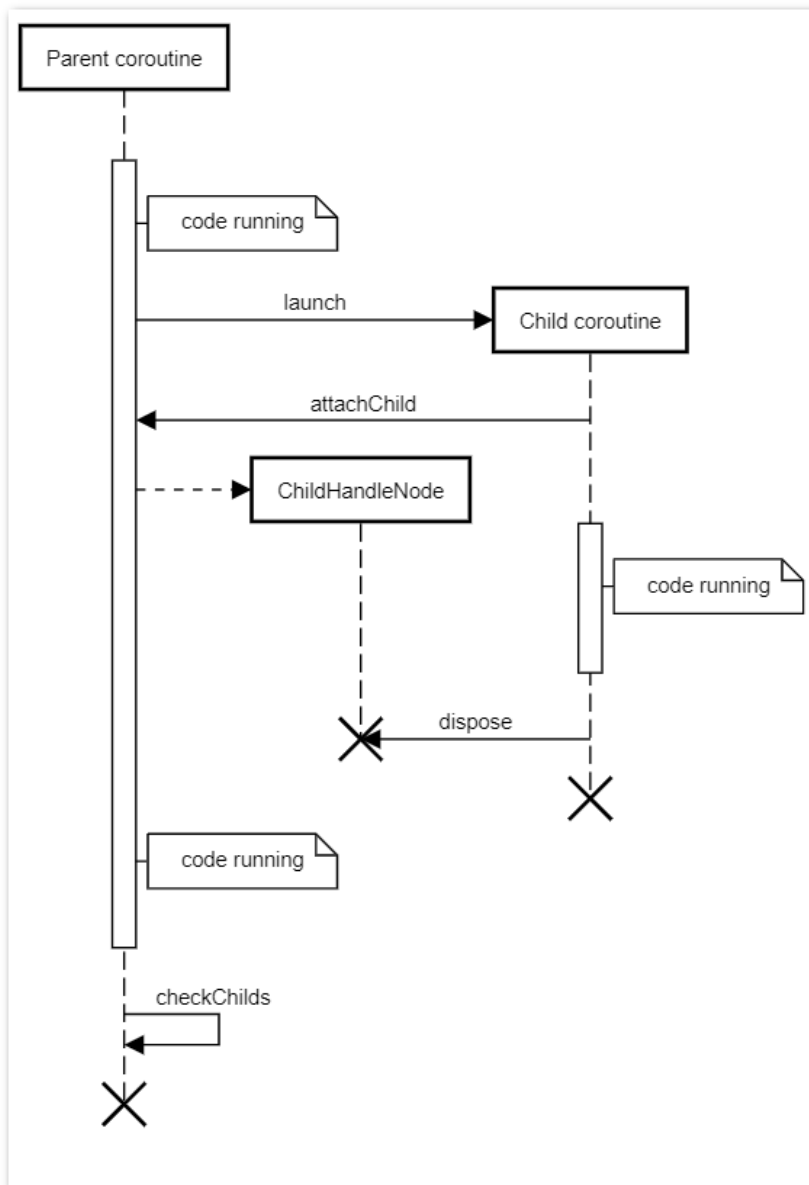
Смотрим, что происходит во время выполнения такого сценария.

Дочерняя корутина при запуске создает связь `ChildHandleNode` и начинает свою работу. Когда ее код выполнен, она разрывает связь, вызывая метод `ChildHandleNode.dispose`. Это приводит к тому, что `ChildHandleNode` удаляется из родителя. Дочерняя корутина переходит в статус `Завершена`.

Когда родительская корутина завершает выполнение своего кода, она проверяет, есть ли у нее какая-либо связь. Т.е. содержит ли `Job` какой-либо объект `ChildHandleNode`. Видит, что такого объекта нет. Это значит, что нет незавершенных дочерних корутин. Ждать никого не надо. Можно переходить в статус `Завершена`.

Я нарисовал схему, на которой изображен вышеописанный процесс. Возможно так будет легче его понять.

Столбец слева - это родительская корутина. Пунктирная линия - это ее время жизни. Прямоугольник в ней - это время выполнения ее кода. Справа - дочерняя корутина. А по центру объект связи.



2. Родительская корутина использует launch для запуска дочерней корутины. Родительская корутина завершит выполнение своего кода раньше, чем дочерняя.

От первого случая отличается тем, что родительская закончит работу раньше дочерней. Как мы уже знаем, она будет ждать завершения дочерней. Рассмотрим, как все это происходит.

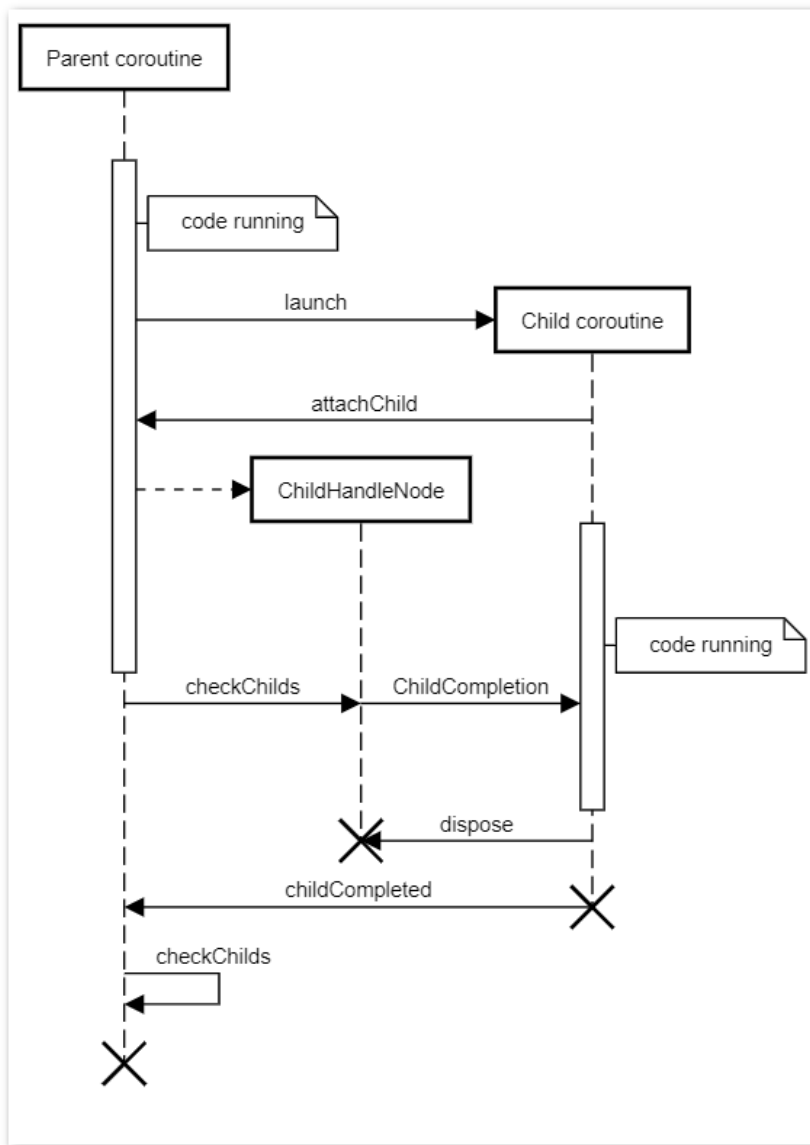
Дочерняя корутина при запуске создает связь ChildHandleNode и начинает свою работу.

Когда родительская корутина завершает выполнение своего кода, она проверяет, есть ли у нее какая-либо связь. Находит объект ChildHandleNode. Значит есть незавершенная дочерняя корутина. Надо ждать ее завершения. Родительская корутина подписывается на окончание дочерней корутины. Для этого она создает объект ChildCompletion и дает дочерней корутине.

Когда дочерняя корутина завершает выполнение своего кода, она разрывает связь ChildHandleNode. Это приводит к тому, что ChildHandleNode удаляется из родителя. Далее дочерняя корутина проверяет, подписывался ли кто на ее завершение. Находит ChildCompletion, который создавал родитель, и использует его для того, чтобы сообщить родительской корутине, что работа дочерней корутины окончена. После этого дочерняя корутина переходит в статус Завершена.

Родительская корутина, получив от дочерней сообщение об окончании ее работы, снова проверяет, есть ли какие-то связи с дочерними корутинами. Ничего не находит и переходит в статус Завершена.

Схема выглядит так:



3. Родительская корутина использует `launch` для запуска пары дочерних корутин. Одна дочерняя корутина завершает выполнение своего кода раньше родительской, а другая позже.

Схему рисовать не буду, т.к. получится слишком сложно. Приведу только краткое описание. Оно представляет собой смесь первого и второго сценария.

Каждая корутина создает свою отдельную связь с родительской (`ChildHandleNode`), и обе начинают выполнять свой код.

Первая корутина заканчивает выполнение своего кода, разрывает свою связь с родителем и переходит в статус Завершена

Родительская корутина заканчивает выполнение своего кода и проверяет связи с дочерними. Находит связь со второй дочерней корутиной и подписывается на ее завершение (`ChildCompletion`).

Вторая дочерняя корутина завершает выполнение своего кода и разрывает связь с родителем. Далее находит `ChildCompletion`, использует его, чтобы сообщить родителю о своем завершении, и переходит в статус Завершена.

Родительская корутина, получив сигнал от второй дочерней, снова проверяет связи с дочерними. Ничего не находит и переходит в статус Завершена.

4. Родительская корутина использует `launch(start = CoroutineStart.LAZY)` для создания дочерней корутины. Но не запускает ее.

В уроке про билдер `launch` мы обсуждали, что корутину можно создать в режиме `Lazy`. Такая корутина начнет выполнять свой код только когда мы явно выполним метод `start` для ее `Job`. Но связь с родителем она установит сразу при создании.

Если мы забудем явно запустить такую дочернюю корутину, то она не начнет выполнение своего кода. А значит она его и не закончит и не разорвет связь с родителем. В итоге родительская корутина никак не сможет перейти в статус Завершена.

Если вдруг эта родительская корутина сама является дочерней и ее родитель вызвал метод `join`, чтобы дождаться ее завершения, то код просто никогда не пойдет дальше этого `join`.

Поэтому будьте внимательны с `Lazy` корутинами.

5. Родительская корутина использует `asunc` для запуска дочерней корутины.

`asunc` корутина обычно возвращает какой-либо результат. Родительская корутина ждет этот результат с помощью метода `await`. Поэтому код дочерней корутины завершится раньше, чем код родительской, и связь будет удалена. Далее родительская корутина завершит свой код и перейдет в статус Завершена.

В общем очень похоже на первый рассмотренный сценарий с `launch`. Но есть и отличия. Они не затрагивают напрямую связь между родительской и дочерней корутинами. Но если вам интересно, чем под капотом `asunc` корутина отличается от `launch`, то я сейчас об этом расскажу.

Когда родительская корутина вызывает `await` у дочерней, возможны два варианта:

1) дочерняя корутина уже выполнила свой код и результат готов

В этом случае все просто: метод `await` сразу возвращает результат работы дочерней корутины. И родительская корутина продолжает выполнение своего кода.

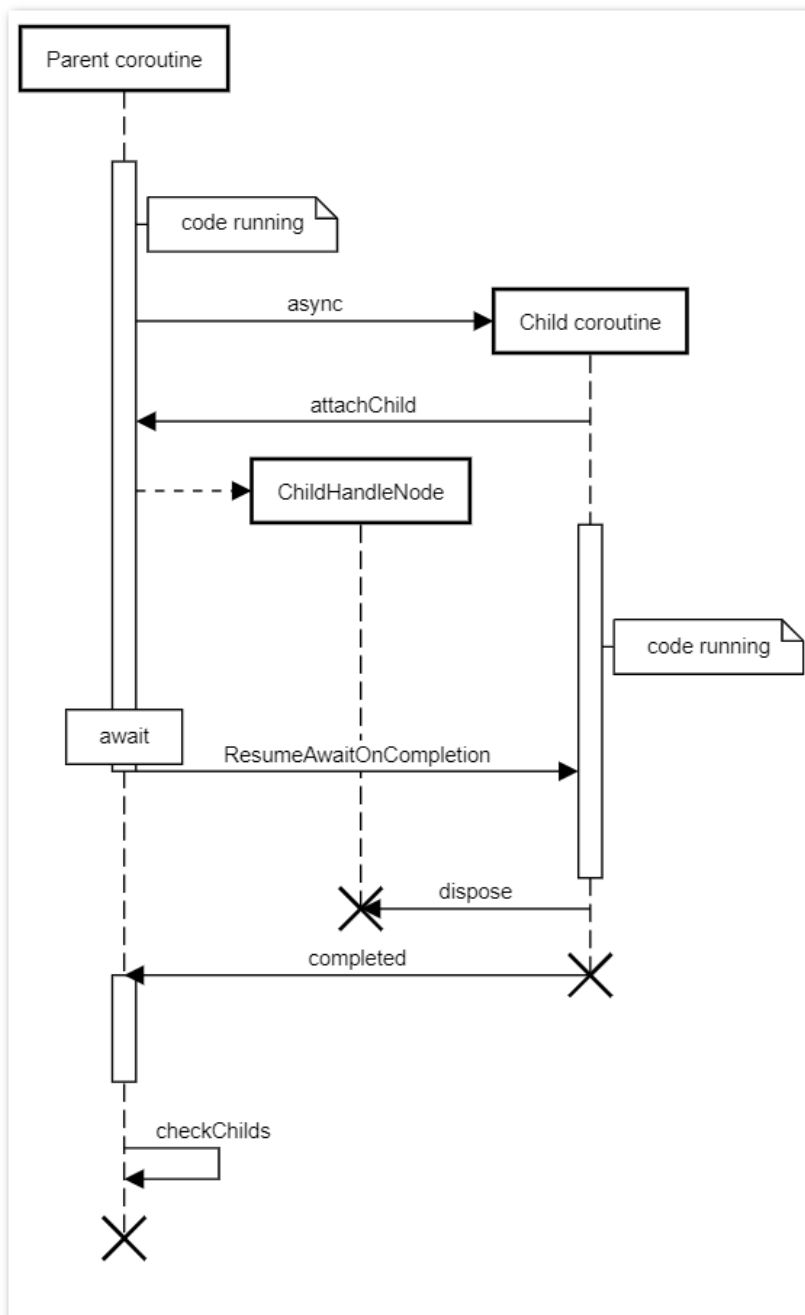
2) дочерняя корутина еще работает и результат надо ждать

Этот случай интереснее. `await` - это `suspend` функция. Она приостанавливает выполнение кода родительской корутины, получает его `Continuation`, упаковывает `Continuation` в объект `ResumeAwaitOnCompletion` и подписывает этот объект на завершение дочерней корутины.

Когда дочерняя корутина завершает выполнение своего кода, она находит объект `ResumeAwaitOnCompletion` и передает ему результат своей работы. `ResumeAwaitOnCompletion` возобновляет выполнение родительского `Continuation` и передает ему результат. Таким образом возобновляется выполнение кода в родительской корутине.

Родительская корутина завершает выполнение своего кода, проверяет, что нет незавершенных дочерних и завершается.

Схема:



Теперь вы знаете, как связаны между собой родительская и дочерняя корутины. Эта связь также используется при обработке ошибок и отмене корутин. Об этом поговорим в следующих уроках.